

TEORETICKÁ INFORMATIKA

Algoritmus DPLL

David Weber
weber3@spsejecna.cz



Rok 2025/26

Konjunktivní normální forma

Konjunktivní normální forma

Definice (Literál a klauzule)

Mějme logické proměnné $x_1, x_2, \dots, x_n$. Pak definujeme následující:

- **Literál** je jakákoliv **proměnná** x_i , **nebo její negace** $\neg x_i$
- **Klauzule** sestává z jednoho nebo více literálů oddělených buď spojkou $\wedge$, nebo $\vee$.

Konjunktivní normální forma

Definice (Literál a klauzule)

Mějme logické proměnné $x_1, x_2, \dots, x_n$. Pak definujeme následující:

- **Literál** je jakákoliv **proměnná** x_i , **nebo její negace** $\neg x_i$
- **Klauzule** sestává z jednoho nebo více literálů oddělených buď spojkou $\wedge$, nebo $\vee$.

Definice (Konjunktivní normální forma)

Mějme logickou formuli φ s proměnnými $x_1, x_2, \dots, x_n$. Pak řekneme, že φ je v **konjunktivní normální formě** (CNF), pokud sestává z klauzulí oddělených spojkou $\wedge$ a literály v každé klauzuli jsou odděleny spojkou $\vee$.

Problémy SAT a UNSAT

Problémy SAT a UNSAT

SPLNITELNOST FORMULE V CNF (SAT)

Vstup: Logická formule φ v CNF.

Otázka: Je formule φ splnitelná?

Problémy SAT a UNSAT

SPLNITELNOST FORMULE V CNF (SAT)

Vstup: Logická formule φ v CNF.

Otázka: Je formule φ splnitelná?

NESPLNITELNOST FORMULE V CNF (UNSAT)

Vstup: Logická formule φ v CNF.

Otázka: Je formule φ nespjitelná?

Algorithmus DPLL

- Algoritmus řešící **SAT** (tzv. **SAT solver**)

- Algoritmus řešící SAT (tzv. **SAT solver**)
- Martin Davis, Hilary Putnam, George Logemann, Donald W. Loveland (1961)

- Algoritmus řešící SAT (tzv. **SAT solver**)
- Martin Davis, Hilary Putnam, George Logemann, Donald W. Loveland (1961)
- Postup založený na rekurzivním „zkracování“ logické formule v CNF.

Jednotková klauzule, polarita a čisté literály

Jednotková klauzule, polarita a čisté literály

Definice (Polarita literálu)

Je-li literál buď proměnná x , nebo její negace $\neg x$, pak říkáme, že

- x má **kladnou polaritu** a
- $\neg x$ má **zápornou polaritu**.

Jednotková klauzule, polarita a čisté literály

Definice (Polarita literálu)

Je-li literál buď proměnná x , nebo její negace $\neg x$, pak říkáme, že

- x má **kladnou polaritu** a
- $\neg x$ má **zápornou polaritu**.

Definice (Jednotková klauzule)

Klauzule se nazývá **jednotková**, pokud obsahuje **právě jeden literál**.

Jednotková klauzule, polarita a čisté literály

Definice (Polarita literálu)

Je-li literál buď proměnná x , nebo její negace $\neg x$, pak říkáme, že

- x má **kladnou polaritu** a
- $\neg x$ má **zápornou polaritu**.

Definice (Jednotková klauzule)

Klauzule se nazývá **jednotková**, pokud obsahuje **právě jeden literál**.

Definice (Čistý literál)

Literál ℓ v logické formuli φ nazveme **čistý**, pokud se v ní vyskytuje **pouze v jedné polaritě**.

Princip algoritmu DPLL

Princip algoritmu DPLL

- Na vstupu máme logickou formuli φ v CNF s proměnnými $x_1, \dots, x_n$.

Princip algoritmu DPLL

- Na vstupu máme logickou formuli φ v CNF s proměnnými $x_1, \dots, x_n$.
- Chceme zjistit, zdali je φ splnitelná.

Princip algoritmu DPLL

- Na vstupu máme logickou formuli φ v CNF s proměnnými $x_1, \dots, x_n$.
- Chceme zjistit, zdali je φ splnitelná.
- Algoritmus **opakovaně** aplikuje na φ postupně dvojici procedur:

Princip algoritmu DPLL

- Na vstupu máme logickou formuli φ v CNF s proměnnými $x_1, \dots, x_n$.
- Chceme zjistit, zdali je φ splnitelná.
- Algoritmus **opakovaně** aplikuje na φ postupně dvojici procedur:
 - **jednotková propagace** (unit propagation) a

Princip algoritmu DPLL

- Na vstupu máme logickou formuli φ v CNF s proměnnými $x_1, \dots, x_n$.
- Chceme zjistit, zdali je φ splnitelná.
- Algoritmus **opakovaně** aplikuje na φ postupně dvojici procedur:
 - **jednotková propagace** (unit propagation) a
 - **eliminace čistých literálů** (pure literal elimination).

Princip algoritmu DPLL

- Na vstupu máme logickou formuli φ v CNF s proměnnými $x_1, \dots, x_n$.
- Chceme zjistit, zdali je φ splnitelná.
- Algoritmus **opakovaně** aplikuje na φ postupně dvojici procedur:
 - **jednotková propagace** (unit propagation) a
 - **eliminace čistých literálů** (pure literal elimination).
- Ze vzniklé formule φ' zkusíme určit, zdali **je či není splnitelná**.

Princip algoritmu DPLL

- Na vstupu máme logickou formuli φ v CNF s proměnnými $x_1, \dots, x_n$.
- Chceme zjistit, zdali je φ splnitelná.
- Algoritmus **opakovaně** aplikuje na φ postupně dvojici procedur:
 - **jednotková propagace** (unit propagation) a
 - **eliminace čistých literálů** (pure literal elimination).
- Ze vzniklé formule φ' zkusíme určit, zdali **je či není splnitelná**.
- Pokud to nepůjde, formuli „vhodně“ upravíme a znovu aplikujeme stejný postup.

Princip algoritmu DPLL

- Na vstupu máme logickou formuli φ v CNF s proměnnými $x_1, \dots, x_n$.
- Chceme zjistit, zdali je φ splnitelná.
- Algoritmus **opakovaně** aplikuje na φ postupně dvojici procedur:
 - **jednotková propagace** (unit propagation) a
 - **eliminace čistých literálů** (pure literal elimination).
- Ze vzniklé formule φ' zkusíme určit, zdali **je či není splnitelná**.
- Pokud to nepůjde, formuli „vhodně“ upravíme a znovu aplikujeme stejný postup.

Má-li být formule φ splnitelná, pak musíme být schopni splnit **každou její klauzuli**.

Jednotková propagace

Jednotková propagace

Nachází-li se ve formuli jednotková klauzule, tj.

$$\dots \wedge (\dots) \wedge (\ell) \wedge (\dots) \wedge \dots,$$

pak nutně musí být $\ell = 1$.

Jednotková propagace

Nachází-li se ve formuli jednotková klauzule, tj.

$$\dots \wedge (\dots) \wedge (\ell) \wedge (\dots) \wedge \dots,$$

pak nutně musí být $\ell = 1$.

- Na vstupu máme formuli φ a nějaký její literál ℓ .

Jednotková propagace

Nachází-li se ve formuli jednotková klauzule, tj.

$$\dots \wedge (\dots) \wedge (\ell) \wedge (\dots) \wedge \dots,$$

pak nutně musí být $\ell = 1$.

- Na vstupu máme formuli φ a nějaký její literál ℓ .
- Pokud je $\ell = x_i$, pak $x_i = 1$. Naopak pokud $\ell = \neg x_i$, pak $x_i = 0$.

Jednotková propagace

Nachází-li se ve formuli jednotková klauzule, tj.

$$\dots \wedge (\dots) \wedge (\ell) \wedge (\dots) \wedge \dots,$$

pak nutně musí být $\ell = 1$.

- Na vstupu máme formuli φ a nějaký její literál ℓ .
- Pokud je $\ell = x_i$, pak $x_i = 1$. Naopak pokud $\ell = \neg x_i$, pak $x_i = 0$.
- V takovém případě můžeme odstranit všechny ostatní klauzule obsahující ℓ .

Jednotková propagace

Nachází-li se ve formuli jednotková klauzule, tj.

$$\dots \wedge (\dots) \wedge (\ell) \wedge (\dots) \wedge \dots,$$

pak nutně musí být $\ell = 1$.

- Na vstupu máme formuli φ a nějaký její literál ℓ .
- Pokud je $\ell = x_i$, pak $x_i = 1$. Naopak pokud $\ell = \neg x_i$, pak $x_i = 0$.
- V takovém případě můžeme odstranit všechny ostatní klauzule obsahující ℓ .

Klauzule ve tvaru $(\dots \vee \ell \vee \dots)$ jsou splněny a dále není třeba je uvažovat (můžeme je odstranit).

Jednotková propagace

Nachází-li se ve formuli jednotková klauzule, tj.

$$\dots \wedge (\dots) \wedge (\ell) \wedge (\dots) \wedge \dots,$$

pak nutně musí být $\ell = 1$.

- Na vstupu máme formuli φ a nějaký její literál ℓ .
- Pokud je $\ell = x_i$, pak $x_i = 1$. Naopak pokud $\ell = \neg x_i$, pak $x_i = 0$.
- V takovém případě můžeme odstranit všechny ostatní klauzule obsahující ℓ .

Klauzule ve tvaru $(\dots \vee \ell \vee \dots)$ jsou splněny a dále není třeba je uvažovat (můžeme je odstranit).

- Naopak pokud nějaká klauzule obsahuje ℓ **v opačné polaritě** (tj. $\neg \ell$), pak jeho hodnota je 0.

Jednotková propagace

Nachází-li se ve formuli jednotková klauzule, tj.

$$\dots \wedge (\dots) \wedge (\ell) \wedge (\dots) \wedge \dots,$$

pak nutně musí být $\ell = 1$.

- Na vstupu máme formuli φ a nějaký její literál ℓ .
- Pokud je $\ell = x_i$, pak $x_i = 1$. Naopak pokud $\ell = \neg x_i$, pak $x_i = 0$.
- V takovém případě můžeme odstranit všechny ostatní klauzule obsahující ℓ .

Klauzule ve tvaru $(\dots \vee \ell \vee \dots)$ jsou splněny a dále není třeba je uvažovat (můžeme je odstranit).

- Naopak pokud nějaká klauzule obsahuje ℓ **v opačné polaritě** (tj. $\neg \ell$), pak jeho hodnota je 0.

Takový literál můžeme z formule odstranit. Hodnota klauzule je závislá pouze na zbylých literálech.

Eliminace čistých literálů

Eliminace čistých literálů

Pokud se vyskytuje nějaký literál ℓ ve formuli φ **pouze v jedné polaritě**, tj.

$$\dots (\dots \vee \ell \vee \dots) \wedge \dots \wedge (\dots \wedge \ell \wedge \dots) \wedge \dots$$

Formule φ tedy neobsahuje $\neg \ell$.

Eliminace čistých literálů

Pokud se vyskytuje nějaký literál ℓ ve formuli φ **pouze v jedné polaritě**, tj.

$$\dots (\dots \vee \ell \vee \dots) \wedge \dots \wedge (\dots \wedge \ell \wedge \dots) \wedge \dots$$

Formule φ tedy neobsahuje $\neg \ell$.

- Na vstupu máme formuli φ a její literál ℓ .

Eliminace čistých literálů

Pokud se vyskytuje nějaký literál ℓ ve formuli φ **pouze v jedné polaritě**, tj.

$$\dots (\dots \vee \ell \vee \dots) \wedge \dots \wedge (\dots \wedge \ell \wedge \dots) \wedge \dots$$

Formule φ tedy neobsahuje $\neg\ell$.

- Na vstupu máme formuli φ a její literál ℓ .
- V takovém případě existuje jednoznačné ohodnocení literálu ℓ takové, že všechny klauzule jej obsahující jsou splněny.

Eliminace čistých literálů

Pokud se vyskytuje nějaký literál ℓ ve formuli φ **pouze v jedné polaritě**, tj.

$$\dots (\dots \vee \ell \vee \dots) \wedge \dots \wedge (\dots \wedge \ell \wedge \dots) \wedge \dots$$

Formule φ tedy neobsahuje $\neg \ell$.

- Na vstupu máme formuli φ a její literál ℓ .
- V takovém případě existuje jednoznačné ohodnocení literálu ℓ takové, že všechny klauzule jej obsahující jsou splněny.
- Resp. pokud $\ell = x_i$, pak nastavíme $x_i = 1$. Jinak pokud $\ell = \neg x_i$, pak nastavíme $x_i = 0$.

Eliminace čistých literálů

Pokud se vyskytuje nějaký literál ℓ ve formuli φ **pouze v jedné polaritě**, tj.

$$\dots (\dots \vee \ell \vee \dots) \wedge \dots \wedge (\dots \wedge \ell \wedge \dots) \wedge \dots$$

Formule φ tedy neobsahuje $\neg \ell$.

- Na vstupu máme formuli φ a její literál ℓ .
- V takovém případě existuje jednoznačné ohodnocení literálu ℓ takové, že všechny klauzule jej obsahující jsou splněny.
- Resp. pokud $\ell = x_i$, pak nastavíme $x_i = 1$. Jinak pokud $\ell = \neg x_i$, pak nastavíme $x_i = 0$.

Takové obsahující ℓ můžeme z φ odstranit,
protože už jsou splněny.

Jak nám procedury pomůžou?



Postup algoritmu

Postup algoritmu

Na vstupu dostaneme formuli φ v CNF.

Postup algoritmu

Na vstupu dostaneme formuli φ v CNF.

- 1 Opakovaně aplikujeme **jednotkovou eliminaci** na formuli φ , dokud v ní existují jednotkové klauzule.

Postup algoritmu

Na vstupu dostaneme formuli φ v CNF.

- 1 Opakovaně aplikujeme **jednotkovou eliminaci** na formuli φ , dokud v ní existují jednotkové klauzule.
- 2 Opakovaně aplikujeme **eliminaci čistých literálů** na formuli φ , dokud v ní existují čisté literály.

Postup algoritmu

Na vstupu dostaneme formuli φ v CNF.

- 1 Opakovaně aplikujeme **jednotkovou eliminaci** na formuli φ , dokud v ní existují jednotkové klauzule.
- 2 Opakovaně aplikujeme **eliminaci čistých literálů** na formuli φ , dokud v ní existují čisté literály.
- 3 Pokud je výsledná formule φ' prázdná (tzn. splnili jsme všechny klauzule), pak **prohlásíme φ za splnitelnou** a skončíme.

Postup algoritmu

Na vstupu dostaneme formuli φ v CNF.

- 1 Opakovaně aplikujeme **jednotkovou eliminaci** na formuli φ , dokud v ní existují jednotkové klauzule.
- 2 Opakovaně aplikujeme **eliminaci čistých literálů** na formuli φ , dokud v ní existují čisté literály.
- 3 Pokud je výsledná formule φ' prázdná (tzn. splnili jsme všechny klauzule), pak **prohlásíme φ za splnitelnou** a skončíme.
- 4 Jinak pokud výsledná formule φ' obsahuje prázdnou klauzuli, pak **prohlásíme φ za nespílitelnou** a skončíme.

Postup algoritmu

Na vstupu dostaneme formuli φ v CNF.

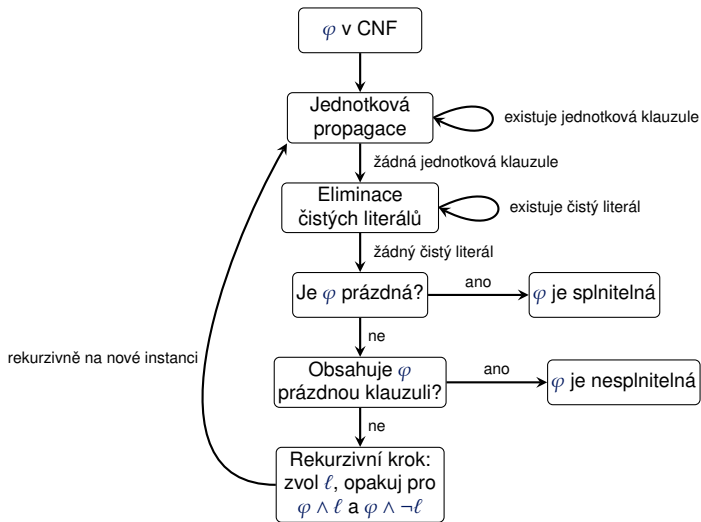
- 1 Opakovaně aplikujeme **jednotkovou eliminaci** na formuli φ , dokud v ní existují jednotkové klauzule.
- 2 Opakovaně aplikujeme **eliminaci čistých literálů** na formuli φ , dokud v ní existují čisté literály.
- 3 Pokud je výsledná formule φ' prázdná (tzn. splnili jsme všechny klauzule), pak **prohlásíme φ za splnitelnou** a skončíme.
- 4 Jinak pokud výsledná formule φ' obsahuje prázdnou klauzuli, pak **prohlásíme φ za nespílitelnou** a skončíme.
- 5 Vybereme náhodně literál ℓ ve formuli φ' a zavoláme rekurzivně DPLL na formule $\varphi' \wedge \ell$ a $\varphi' \wedge \neg \ell$.

Postup algoritmu

Na vstupu dostaneme formuli φ v CNF.

- 1 Opakovaně aplikujeme **jednotkovou eliminaci** na formuli φ , dokud v ní existují jednotkové klauzule.
- 2 Opakovaně aplikujeme **eliminaci čistých literálů** na formuli φ , dokud v ní existují čisté literály.
- 3 Pokud je výsledná formule φ' prázdná (tzn. splnili jsme všechny klauzule), pak **prohlásíme φ za splnitelnou** a skončíme.
- 4 Jinak pokud výsledná formule φ' obsahuje prázdnou klauzuli, pak **prohlásíme φ za nespjitelnou** a skončíme.
- 5 Vybereme náhodně literál ℓ ve formuli φ' a zavoláme rekurzivně DPLL na formule $\varphi' \wedge \ell$ a $\varphi' \wedge \neg \ell$.
- 6 Je-li $\varphi' \wedge \ell$ nebo $\varphi' \wedge \neg \ell$ splnitelná, je i φ splnitelná.

Znázornění algoritmu



Algoritmus: DPLL

Vstup: Formule φ v CNF

- 1 **while** existuje jednotková klauzule (ℓ) ve formuli φ **do**
 - 2 $\varphi \leftarrow \text{UNITPROPAGATION}(\ell, \varphi)$
 - 3 **while** existuje literál ℓ v jediné polaritě ve φ **do**
 - 4 $\varphi \leftarrow \text{PURELITERALELIMINATION}(\ell, \varphi)$
 // Všechny klauzule jsou pravdivé
 - 5 **if** je φ prázdná **then return** 1;
 // Všechny literály v klauzuli jsou nepravdivé
 - 6 **if** obsahuje φ prázdnou klauzuli **then return** 0;
 // Zkusíme ohodnocení pro $\ell = 0$ a $\ell = 1$
 - 7 Vyber náhodně literál ℓ
 - 8 **return** $\text{DPLL}(\varphi \wedge \ell) \vee \text{DPLL}(\varphi \wedge \neg \ell)$
-

Mějme zadanou formuli

$$\varphi = (a) \wedge (\neg a \vee b) \wedge (e \vee c) \wedge (c \vee d) \wedge (\neg c \vee d) \wedge (c \vee \neg d).$$

Algoritmus DPLL

Příklad

Krok 1: start

Pracovní formule

(a)

$(\neg a \vee b)$

$(e \vee c)$

$(c \vee d)$

$(\neg c \vee d)$

$(c \vee \neg d)$

Rekurze



Aktuální ohodnocení:

žádné přiřazení

Komentář:

Jednotková klauzule (a)
spustí
UNITPROPAGATION.

Krok 2: po UNITPROPAGATION

Pracovní formule

$$(a)$$

$$(\neg a \vee b)$$

$$(e \vee c)$$

$$(c \vee d)$$

$$(\neg c \vee d)$$

$$(c \vee \neg d)$$

Rekurze



Aktuální ohodnocení:

$$a = 1, b = 1$$

Komentář:

Po propagaci už nezůstává žádná jednotková klauzule.

Literál e je čistý.

Algoritmus DPLL

Příklad

Krok 3: po PURELITERALELIMINATION

Pracovní formule

(a)

$(\neg a \vee b)$

$(e \vee c)$

$(c \vee d)$

$(\neg c \vee d)$

$(c \vee \neg d)$

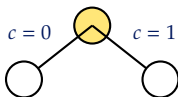
Aktuální ohodnocení:

$a = 1, b = 1, e = 1$

Komentář:

Nejsou jednotkové
klauzule ani čisté literály.
Větvíme podle proměnné
 c .

Rekurze



Krok 4: větev $c = 0$

Pracovní formule

(a)

$(\neg a \vee b)$

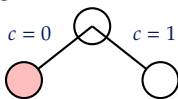
$(e \vee c)$

(d)

$(\neg c \vee d)$

$(\neg d)$

Rekurze



Aktuální ohodnocení:

$a = 1, b = 1, e = 1, c = 0$

Komentář:

Po dosazení $c = 0$ vzniknou klauzule (d) a $(\neg d)$, tedy konflikt. Tato větev je nespílitelná.

Algoritmus DPLL

Příklad

Krok 5: větev $c = 1$

Pracovní formule

(a)

$(\neg a \vee b)$

$(e \vee c)$

$(c \vee d)$

$(\neg c \vee d)$

$(c \vee \neg d)$

Aktuální ohodnocení:

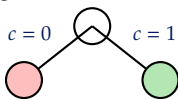
$a = 1, b = 1, e = 1, c = 1, d = 1$

Komentář:

Po dosazení $c = 1$ vznikne jednotková klauzule (d) .

Po UNITPROPAGATION dostaneme prázdnou formuli, tedy φ je splnitelná.

Rekurze



- Nejhorší případ nastane, pokud při žádném volání **DPLL** neexituje v příslušné formuli **žádná jednotková klauzule** vyjma té, kterou jsme vytvořili při sestavení formule $\varphi \wedge \ell$, resp. $\varphi \wedge \neg \ell$, ani **čistý literál**.

- Nejhorší případ nastane, pokud při žádném volání **DPLL** neexituje v příslušné formuli **žádná jednotková klauzule** vyjma té, kterou jsme vytvořili při sestavení formule $\varphi \wedge \ell$, resp. $\varphi \wedge \neg \ell$, ani **čistý literál**.
- Máme-li formuli ve tvaru $\varphi \wedge \ell$, resp. $\varphi \wedge \neg \ell$, pak literál ℓ eliminujeme v dalším volání a všechny ostatní zůstanou zachované.

- Nejhorší případ nastane, pokud při žádném volání **DPLL** neexituje v příslušné formuli **žádná jednotková klauzule** vyjma té, kterou jsme vytvořili při sestavení formule $\varphi \wedge \ell$, resp. $\varphi \wedge \neg \ell$, ani **čistý literál**.
- Máme-li formuli ve tvaru $\varphi \wedge \ell$, resp. $\varphi \wedge \neg \ell$, pak literál ℓ eliminujeme v dalším volání a všechny ostatní zůstanou zachované.

Formule se tedy zkrátí pouze o **jeden literál**.

- Nejhorší případ nastane, pokud při žádném volání **DPLL** neexistuje v příslušné formuli **žádná jednotková klauzule** vyjma té, kterou jsme vytvořili při sestavení formule $\varphi \wedge \ell$, resp. $\varphi \wedge \neg \ell$, ani **čistý literál**.
- Máme-li formuli ve tvaru $\varphi \wedge \ell$, resp. $\varphi \wedge \neg \ell$, pak literál ℓ eliminujeme v dalším volání a všechny ostatní zůstanou zachované.

Formule se tedy zkrátí pouze o **jeden literál**.

- Označme si n počet literálů vstupní formule φ .

Časová složitost DPLL

- Nejhorší případ nastane, pokud při žádném volání **DPLL** neexistuje v příslušné formuli **žádná jednotková klauzule** vyjma té, kterou jsme vytvořili při sestavení formule $\varphi \wedge \ell$, resp. $\varphi \wedge \neg \ell$, ani **čistý literál**.
- Máme-li formuli ve tvaru $\varphi \wedge \ell$, resp. $\varphi \wedge \neg \ell$, pak literál ℓ eliminujeme v dalším volání a všechny ostatní zůstanou zachované.

Formule se tedy zkrátí pouze o **jeden literál**.

- Označme si n počet literálů vstupní formule φ .
- Dále si označme:

Časová složitost DPLL

- Nejhorší případ nastane, pokud při žádném volání **DPLL** neexituje v příslušné formuli **žádná jednotková klauzule** vyjma té, kterou jsme vytvořili při sestavení formule $\varphi \wedge \ell$, resp. $\varphi \wedge \neg \ell$, ani **čistý literál**.
- Máme-li formuli ve tvaru $\varphi \wedge \ell$, resp. $\varphi \wedge \neg \ell$, pak literál ℓ eliminujeme v dalším volání a všechny ostatní zůstanou zachované.

Formule se tedy zkrátí pouze o **jeden literál**.

- Označme si n počet literálů vstupní formule φ .
- Dále si označme:
 - $T(n)$ = počet kroků algoritmu nad formulí o n literálech,

Časová složitost DPLL

- Nejhorší případ nastane, pokud při žádném volání **DPLL** neexituje v příslušné formuli **žádná jednotková klauzule** vyjma té, kterou jsme vytvořili při sestavení formule $\varphi \wedge \ell$, resp. $\varphi \wedge \neg \ell$, ani **čistý literál**.
- Máme-li formuli ve tvaru $\varphi \wedge \ell$, resp. $\varphi \wedge \neg \ell$, pak literál ℓ eliminujeme v dalším volání a všechny ostatní zůstanou zachované.

Formule se tedy zkrátí pouze o **jeden literál**.

- Označme si n počet literálů vstupní formule φ .
- Dále si označme:
 - $T(n)$ = počet kroků algoritmu nad formulí o n literálech,
 - $r(n)$ = režie jednoho volání.

Časová složitost DPLL

- V jednom volání DPLL strávíme $r(n)$ kroků eliminací **jednotkových klauzulí** a **čistých literálů**.

Časová složitost DPLL

- V jednom volání DPLL strávíme $r(n)$ kroků eliminací **jednotkových klauzulí** a **čistých literálů**.
- Na konci voláme dvakrát DPLL na vstup velikosti $n - 1$, čímž strávíme $2 \cdot T(n - 1)$ kroků.

Časová složitost DPLL

- V jednom volání DPLL strávíme $r(n)$ kroků eliminací **jednotkových klauzulí** a **čistých literálů**.
- Na konci voláme dvakrát DPLL na vstup velikosti $n - 1$, čímž strávíme $2 \cdot T(n - 1)$ kroků.
- Můžeme předpokládat, že režii strávíme polynomiální čas, tzn. $r(n) = O(n^k)$, kde $k \in \mathbb{R}^+$.

Časová složitost DPLL

- V jednom volání DPLL strávíme $r(n)$ kroků eliminací **jednotkových klauzulí** a **čistých literálů**.
- Na konci voláme dvakrát DPLL na vstup velikosti $n - 1$, čímž strávíme $2 \cdot T(n - 1)$ kroků.
- Můžeme předpokládat, že režii strávíme polynomiální čas, tzn. $r(n) = O(n^k)$, kde $k \in \mathbb{R}^+$.
- Tím dostáváme následující rovnici:

$$T(n) = 2 \cdot T(n - 1) + r(n) = 2 \cdot T(n - 1) + O(n^k).$$

Časová složitost DPLL

- V jednom volání DPLL strávíme $r(n)$ kroků eliminací **jednotkových klauzulí** a **čistých literálů**.
- Na konci voláme dvakrát DPLL na vstup velikosti $n - 1$, čímž strávíme $2 \cdot T(n - 1)$ kroků.
- Můžeme předpokládat, že režii strávíme polynomiální čas, tzn. $r(n) = O(n^k)$, kde $k \in \mathbb{R}^+$.
- Tím dostáváme následující rovnici:

$$T(n) = 2 \cdot T(n - 1) + r(n) = 2 \cdot T(n - 1) + O(n^k).$$

Tzv. **rekurentní rovnice**, kdy hodnota zkoumané funkce závisí na „sobě samotné“.

Časová složitost DPLL

- V jednom volání DPLL strávíme $r(n)$ kroků eliminací **jednotkových klauzulí** a **čistých literálů**.
- Na konci voláme dvakrát DPLL na vstup velikosti $n - 1$, čímž strávíme $2 \cdot T(n - 1)$ kroků.
- Můžeme předpokládat, že režii strávíme polynomiální čas, tzn. $r(n) = O(n^k)$, kde $k \in \mathbb{R}^+$.
- Tím dostáváme následující rovnici:

$$T(n) = 2 \cdot T(n - 1) + r(n) = 2 \cdot T(n - 1) + O(n^k).$$

Tzv. **rekurentní rovnice**, kdy hodnota zkoumané funkce závisí na „sobě samotné“.

- Lze ukázat, že $T(n) = O(2^n)$. Algoritmus DPLL je tedy v nejhorším případě exponenciální.

- STUART J. RUSSELL a NORVIG, Peter. *Artificial intelligence: A modern approach*. 3rd ed. Englewood Cliffs, N.J: Pearson, 2016. ISBN 8120323823.